Code Reference: main.cjs

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	main.cjs
Purpose	Electron main process: creates the desktop window, starts the local backend, handles menus, update handoff, and application lifecycle.
Lines	453
Size	15,134 bytes
Talks to	builder frontend, local backend, public website runtime, portfolio catalog, Cloudflare/AI layer, GitHub/release layer
Functions	21
Variables/constants	42
API mentions	0
Signals	43

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 3	const fs = require("node:fs");
Line 4	const fsp = require("node:fs/promises");
Line 5	const net = require("node:net");
Line 6	const path = require("node:path");

Important implementation signals

Item	Explanation
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const { execFile } = require("node:child_process");
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: const fs = require("node:fs");
Compiler or simulator at line 4	Part of the Compile Code workspace or language/tool detection. Evidence: const fsp = require("node:fs/promises");
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: const net = require("node:net");
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: const path = require("node:path");
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: const { pathToFileURL } = require("node:url");
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: const { promisify } = require("node:util");
Process execution at line 13	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
Security or auth at line 27	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers"
Cloudflare or AI at line 38	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: const directoryRefreshes = ["cloudflare"];
Security or auth at line 48	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Git command at line 53	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,

Item	Explanation
Git command at line 54	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 55	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 56	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 57	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 58	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe"
Installer at line 61	Windows setup, update, shortcut, or installation behavior. Evidence: function dispatchBuilderMenuAction(action) {
Installer at line 110	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({ type: "new-project" }) },
Installer at line 111	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target" }) },
Installer at line 113	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type: "save-draft" }) },
Installer at line 114	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () => dispatchBuilderMenuAction({ type: "apply-site" }) },
Rich text at line 127	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: { role: "paste" },
Installer at line 141	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview" }) },
Installer at line 142	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
Installer at line 154	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Open Preferences", accelerator: "CmdOrCtrl+,", click: () => dispatchBuilderMenuAction({ type: "preferences" }) },
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light" }) },
Installer at line 157	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark" }) }
Installer at line 163	Windows setup, update, shortcut, or installation behavior. Evidence: { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type: "builder-guide" }) },
Network request at line 164	Frontend-backend communication or public web/API calls. Evidence: { label: "GitHub Releases", click: () => shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
File write at line 190	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
File write at line 191	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.copyFile(source, target);
File write at line 196	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(target, { recursive: true });
File write at line 212	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(path.dirname(target), { recursive: true });
Git command at line 241	Repository state, publishing, branch checks, or release automation. Evidence: throw lastError || new Error("Git executable was not found.");
Git command at line 250	Repository state, publishing, branch checks, or release automation. Evidence: return fileExists(path.join(workspaceRoot, ".git"));
File write at line 267	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });
Installer at line 279	Windows setup, update, shortcut, or installation behavior. Evidence: const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
Installer at line 280	Windows setup, update, shortcut, or installation behavior. Evidence: ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
File write at line 286	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(workspaceRoot, { recursive: true });
File write at line 287	Writes drafts, generated portfolio output, compiler artifacts, uploaded files, or installer data. Evidence: await fsp.mkdir(portfolioRoot, { recursive: true });

Item	Explanation
Network request at line 363	Frontend-backend communication or public web/API calls. Evidence: return `http://127.0.0.1:${port}`;
Rich text at line 377	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: backgroundColor: "#eef8fd",

Important constants and variables

Item	Explanation
fs	Line 3: const fs = require("node:fs");
fsp	Line 4: const fsp = require("node:fs/promises");
net	Line 5: const net = require("node:net");
path	Line 6: const path = require("node:path");
mainWindow	Line 10: let mainWindow = null;
builderOrigin	Line 11: let builderOrigin = "";
updateShutdownStarted	Line 12: let updateShutdownStarted = false;
execFileAsync	Line 13: const execFileAsync = promisify(execFile);
defaultRepositoryUrl	Line 14: const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY || "";
defaultWorkspaceHomeName	Line 15: const defaultWorkspaceHomeName = "OMB Portfolio Builder";
rootFilesToRefresh	Line 17: const rootFilesToRefresh = [
rootFilesToSeed	Line 30: const rootFilesToSeed = [
directoryRefreshes	Line 38: const directoryRefreshes = ["cloudflare"];
directorySeeds	Line 39: const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];
portfolioFilesToSeed	Line 40: const portfolioFilesToSeed = [
portfolioDirectoriesToSeed	Line 51: const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];
gitCandidates	Line 52: const gitCandidates = [
payload	Line 63: const payload = JSON.stringify(action);
shouldFullscreen	Line 98: const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();
entries	Line 197: const entries = await fsp.readdir(source, { withFileTypes: true });
sourcePath	Line 199: const sourcePath = path.join(source, entry.name);
targetPath	Line 200: const targetPath = path.join(target, entry.name);
entries	Line 218: const entries = await fsp.readdir(directory);
lastError	Line 226: let lastError = null;
current	Line 254: let current = path.dirname(process.execPath);
next	Line 257: const next = path.dirname(current);
bundledSiteRoot	Line 278: const bundledSiteRoot = path.join(process.resourcesPath, "site");
defaultWorkspaceHome	Line 279: const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
workspaceHome	Line 282: const workspaceHome = process.env.OMB_WORKSPACE_HOME || defaultWorkspaceHome;
workspaceRoot	Line 283: const workspaceRoot = path.join(workspaceHome, "builder");
portfolioRoot	Line 284: const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE || path.join(workspaceHome, "portfolio");
envWorkspace	Line 315: const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;
workspaceRoot	Line 323: const workspaceRoot = path.resolve(__dirname);
nearbyRepository	Line 329: const nearbyRepository = findRepositoryNearExecutable();
tester	Line 341: const tester = net.createServer();

Item	Explanation
address	Line 344: const address = tester.address();
port	Line 345: const port = typeof address === "object" && address ? address.port : 0;
port	Line 352: const port = await findFreePort();
serverPath	Line 361: const serverPath = path.join(workspaceRoot, "server.mjs");
iconPath	Line 367: const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
refreshFullscreenMenu	Line 388: const refreshFullscreenMenu = () => {
gotLock	Line 426: const gotLock = app.requestSingleInstanceLock();

Function-by-function detail

dispatchBuilderMenuAction (main.cjs, lines 61-68)

dispatchBuilderMenuAction part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 61-68 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, stringify, executeJavaScript, dispatchEvent, and CustomEvent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 62: if (!mainWindow || mainWindow.isDestroyed()) return;.

Important local values include payload at line 63. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
61: function dispatchBuilderMenuAction(action) {
62:   if (!mainWindow || mainWindow.isDestroyed()) return;
63:   const payload = JSON.stringify(action);
64:   mainWindow.webContents.executeJavaScript(
65:     `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
66:     true
67:   ).catch(() => {});
68: }
```

quitForBuilderUpdate (main.cjs, lines 70-92)

quitForBuilderUpdate part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 70-92 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, destroy, quit, and exit. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 71: if (updateShutdownStarted) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
70: function quitForBuilderUpdate() {
71:   if (updateShutdownStarted) return;
72:   updateShutdownStarted = true;
73:   try {
74:     if (mainWindow && !mainWindow.isDestroyed()) {
75:       mainWindow.destroy();
76:     }
77:   } catch {
78:     // The process exit fallback below still guarantees the updater can continue.
79:   }
80:   try {
81:     app.quit();
82:   } catch {
83:     // Fall through to app.exit.
```

```
  84:   }
  85:   setTimeout(() => {
  86:     try {
  87:       app.exit(0);
  88:     } catch {
  89:       process.exit(0);
  90:     }
  91:   }, 1500);
  92: }
```

setBuilderFullScreen (main.cjs, lines 96-103)

setBuilderFullScreen part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 96-103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, isFullScreen, setFullScreen, setMenuBarVisibility, setAutoHideMenuBar, setMenu, and createAppMenu. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 97: if (!mainWindow || mainWindow.isDestroyed()) return;.

Important local values include shouldFullscreen at line 98. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  96: function setBuilderFullScreen(nextState = null) {
  97:   if (!mainWindow || mainWindow.isDestroyed()) return;
  98:   const shouldFullscreen = typeof nextState === "boolean" ? nextState : !mainWindow.isFullScreen();
  99:   mainWindow.setFullScreen(shouldFullscreen);
 100:   mainWindow.setMenuBarVisibility(true);
 101:   mainWindow.setAutoHideMenuBar(false);
 102:   mainWindow.setMenu(createAppMenu(builderOrigin));
 103: }
```

createAppMenu (main.cjs, lines 105-184)

createAppMenu creates an object, ui structure, process, payload, or generated artifact. In this file, it appears around lines 105-184 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references buildFromTemplate, dispatchBuilderMenuAction, setBuilderFullScreen, openExternal, isDestroyed, isMinimized, restore, focus, fileExists, and accessSync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 106: return Menu.buildFromTemplate([; line 168: if (!mainWindow || mainWindow.isDestroyed()) return;; line 181: return true;; line 183: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
 105: function createAppMenu(origin) {
 106:   return Menu.buildFromTemplate([
 107:     {
 108:       label: "File",
 109:       submenu: [
 110:         { label: "New Project", accelerator: "CmdOrCtrl+N", click: () => dispatchBuilderMenuAction({
type: "new-project" }) },
 111:         { label: "Publishing Target", click: () => dispatchBuilderMenuAction({ type: "publishing-target"
}) },
 112:         { type: "separator" },
 113:         { label: "Save Draft", accelerator: "CmdOrCtrl+S", click: () => dispatchBuilderMenuAction({ type:
 "save-draft" }) },
 114:         { label: "Apply to Site", accelerator: "CmdOrCtrl+Shift+S", click: () =>
dispatchBuilderMenuAction({ type: "apply-site" }) },
 115:         { type: "separator" },
 116:         { role: "quit", label: "Exit" }
 117:       ]
 118:     },
 119:     {
 120:       label: "Edit",
 121:       submenu: [
 122:         { role: "undo" },
 123:         { role: "redo" },
 124:         { type: "separator" },
```

```
  125:         { role: "cut" },
  126:         { role: "copy" },
  127:         { role: "paste" },
  128:         { role: "selectAll" }
  129:       ]
  130:     },
  131:     {
  132:       label: "View",
  133:       submenu: [
  134:         { role: "reload" },
  135:         { role: "forceReload" },
  136:         { type: "separator" },
  137:         { role: "resetZoom" },
  138:         { role: "zoomIn" },
  139:         { role: "zoomOut" },
  140:         { type: "separator" },
  141:         { label: "Portfolio Preview", click: () => dispatchBuilderMenuAction({ type: "portfolio-preview"
}) },
  142:         { label: "Check Updates", click: () => dispatchBuilderMenuAction({ type: "check-updates" }) },
  143:         { type: "separator" },
  144:         {
  145:           label: mainWindow?.isFullScreen?.() ? "Exit Full Screen" : "Toggle Full Screen",
  146:           accelerator: "F11",
  147:           click: () => setBuilderFullScreen()
  148:         }
  149:       ]
  150:     },
  151:     {
  152:       label: "Preferences",
  153:       submenu: [
  154:         { label: "Open Preferences", accelerator: "CmdOrCtrl+,", click: () => dispatchBuilderMenuAction({
 type: "preferences" }) },
  155:         { type: "separator" },
  156:         { label: "Light Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "light"
 }) },
  157:         { label: "Dark Mode", click: () => dispatchBuilderMenuAction({ type: "set-theme", theme: "dark"
}) }
  158:       ]
  159:     },
  160:     {
  161:       label: "Help",
  162:       submenu: [
  163:         { label: "Builder Guide", accelerator: "F1", click: () => dispatchBuilderMenuAction({ type:
"builder-guide" }) },
  164:         { label: "GitHub Releases", click: () =>
shell.openExternal("https://github.com/otienomaurice/omb-portfolio-builder/releases/latest") },
  165:         {
  166:           label: "Focus Builder Window",
  167:           click: () => {
  168:             if (!mainWindow || mainWindow.isDestroyed()) return;
  169:             if (mainWindow.isMinimized()) mainWindow.restore();
  170:             mainWindow.focus();
  171:           }
  172:         }
  173:       ]
  174:     }
  175:   ]);
  176: }
  177:
  178: function fileExists(filePath) {
  179:   try {
  180:     fs.accessSync(filePath);
  181:     return true;
  182:   } catch {
  183:     return false;
  184:   }
```

fileExists (main.cjs, lines 178-185)

fileExists part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 178-185 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references accessSync. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 181: return true;; line 183: return false;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  178: function fileExists(filePath) {
  179:   try {
```

```
180:     fs.accessSync(filePath);
181:     return true;
182:   } catch {
183:     return false;
184:   }
185: }
```

copyFileIfAvailable (main.cjs, lines 187-192)

copyFileIfAvailable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 187-192 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, mkdir, dirname, and copyFile. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 188: if (!fileExists(source)) return;; line 189: if (!overwrite && fileExists(target)) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
187: async function copyFileIfAvailable(source, target, overwrite = false) {
188:   if (!fileExists(source)) return;
189:   if (!overwrite && fileExists(target)) return;
190:   await fsp.mkdir(path.dirname(target), { recursive: true });
191:   await fsp.copyFile(source, target);
192: }
```

copyDirectoryMissingFiles (main.cjs, lines 194-207)

copyDirectoryMissingFiles part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 194-207 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, mkdir, readdir, join, isDirectory, isFile, and copyFileIfAvailable. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 195: if (!fileExists(source)) return;.

Important local values include entries at line 197; sourcePath at line 199; targetPath at line 200. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
194: async function copyDirectoryMissingFiles(source, target) {
195:   if (!fileExists(source)) return;
196:   await fsp.mkdir(target, { recursive: true });
197:   const entries = await fsp.readdir(source, { withFileTypes: true });
198:   for (const entry of entries) {
199:     const sourcePath = path.join(source, entry.name);
200:     const targetPath = path.join(target, entry.name);
201:     if (entry.isDirectory()) {
202:       await copyDirectoryMissingFiles(sourcePath, targetPath);
203:     } else if (entry.isFile()) {
204:       await copyFileIfAvailable(sourcePath, targetPath, false);
205:     }
206:   }
207: }
```

copyDirectoryRefresh (main.cjs, lines 209-214)

copyDirectoryRefresh part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 209-214 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, rm, mkdir, dirname, and cp. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 210: if (!fileExists(source)) return;.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
209: async function copyDirectoryRefresh(source, target) {
210:   if (!fileExists(source)) return;
211:   await fsp.rm(target, { recursive: true, force: true });
212:   await fsp.mkdir(path.dirname(target), { recursive: true });
213:   await fsp.cp(source, target, { recursive: true, force: true });
214: }
```

directoryIsEmpty (main.cjs, lines 216-223)

directoryIsEmpty part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 216-223 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references readdir. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 219: return entries.length === 0;; line 221: return true;.

Important local values include entries at line 218. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
216: async function directoryIsEmpty(directory) {
217:   try {
218:     const entries = await fsp.readdir(directory);
219:     return entries.length === 0;
220:   } catch {
221:     return true;
222:   }
223: }
```

runGit (main.cjs, lines 225-242)

runGit part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 225-242 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references execFileAsync, and Error. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 234: return true;.

Important local values include lastError at line 226. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
225: async function runGit(args, cwd) {
226:   let lastError = null;
227:   for (const candidate of gitCandidates) {
228:     try {
229:       await execFileAsync(candidate, args, {
230:         cwd,
231:         maxBuffer: 10 * 1024 * 1024,
232:         windowsHide: true
233:       });
234:       return true;
235:     } catch (error) {
236:       lastError = error;
237:       if (error.code === "ENOENT") continue;
238:       throw error;
239:     }
240:   }
241:   throw lastError || new Error("Git executable was not found.");
242: }
```

workspaceLooksUsable (main.cjs, lines 244-247)

workspaceLooksUsable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 244-247 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 245: return fileExists(path.join(workspaceRoot, "server.mjs")).

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
244: function workspaceLooksUsable(workspaceRoot) {
245:   return fileExists(path.join(workspaceRoot, "server.mjs"))
246:     && fileExists(path.join(workspaceRoot, "index.html"));
247: }
```

workspaceIsGitBacked (main.cjs, lines 249-251)

workspaceIsGitBacked part of publishing, git, target repository, or authorization behavior. In this file, it appears around lines 249-251 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references fileExists, and join. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 250: return fileExists(path.join(workspaceRoot, ".git"));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
249: function workspaceIsGitBacked(workspaceRoot) {
250:   return fileExists(path.join(workspaceRoot, ".git"));
251: }
```

findRepositoryNearExecutable (main.cjs, lines 253-262)

findRepositoryNearExecutable part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 253-262 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references dirname, workspaceLooksUsable, and workspaceIsGitBacked. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 256: if (workspaceLooksUsable(current) && workspaceIsGitBacked(current)) return current;; line 261: return "";.

Important local values include current at line 254; next at line 257. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253: function findRepositoryNearExecutable() {
254:   let current = path.dirname(process.execPath);
255:   for (let index = 0; index < 8; index += 1) {
256:     if (workspaceLooksUsable(current) && workspaceIsGitBacked(current)) return current;
257:     const next = path.dirname(current);
258:     if (next === current) break;
259:     current = next;
260:   }
261:   return "";
262: }
```

cloneWorkspaceIfPossible (main.cjs, lines 264-275)

cloneWorkspaceIfPossible part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 264-275 and is part of the repository root. usually an entry file, app config, public website file, package

manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references workspaceIsGitBacked, mkdir, directoryIsEmpty, runGit, and dirname. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 5 return statement(s). The most important visible returns are: line 265: if (!defaultRepositoryUrl) return false;; line 266: if (workspaceIsGitBacked(workspaceRoot)) return true;; line 268: if (!(await directoryIsEmpty(workspaceRoot))) return false;; line 271: return workspaceIsGitBacked(workspaceRoot);. There are 1 additional return statements in the function body.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
  264: async function cloneWorkspaceIfPossible(workspaceRoot) {
  265:   if (!defaultRepositoryUrl) return false;
  266:   if (workspaceIsGitBacked(workspaceRoot)) return true;
  267:   await fsp.mkdir(workspaceRoot, { recursive: true });
  268:   if (!(await directoryIsEmpty(workspaceRoot))) return false;
  269:   try {
  270:     await runGit(["clone", "--depth", "1", defaultRepositoryUrl, workspaceRoot],
path.dirname(workspaceRoot));
  271:     return workspaceIsGitBacked(workspaceRoot);
  272:   } catch {
  273:     return false;
  274:   }
  275: }
```

preparePackagedWorkspace (main.cjs, lines 277-312)

This function prepares the writable workspace that the packaged desktop application will use at runtime. The installed app cannot safely treat packaged resources as editable source files, so this function copies or refreshes the required builder and portfolio resources into a location the user can actually write to.

Without this function, a fresh install could open with missing website files, stale resources, or files trapped inside the packaged app bundle. It is one of the reasons the builder can behave like a normal desktop app while still using web assets internally.

It calls or references join, getPath, mkdir, cloneWorkspaceIfPossible, copyFileIfAvailable, copyDirectoryRefresh, and copyDirectoryMissingFiles. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 311: return { workspaceRoot, portfolioRoot };.

Important local values include bundledSiteRoot at line 278; defaultWorkspaceHome at line 279; workspaceHome at line 282; workspaceRoot at line 283; portfolioRoot at line 284. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  277: async function preparePackagedWorkspace() {
  278:   const bundledSiteRoot = path.join(process.resourcesPath, "site");
  279:   const defaultWorkspaceHome = process.platform === "win32" && process.env.LOCALAPPDATA
  280:     ? path.join(process.env.LOCALAPPDATA, defaultWorkspaceHomeName)
  281:     : path.join(app.getPath("userData"), "workspace");
  282:   const workspaceHome = process.env.OMB_WORKSPACE_HOME || defaultWorkspaceHome;
  283:   const workspaceRoot = path.join(workspaceHome, "builder");
  284:   const portfolioRoot = process.env.OMB_PORTFOLIO_WORKSPACE || path.join(workspaceHome, "portfolio");
  285:
  286:   await fsp.mkdir(workspaceRoot, { recursive: true });
  287:   await fsp.mkdir(portfolioRoot, { recursive: true });
  288:   await cloneWorkspaceIfPossible(portfolioRoot);
  289:
  290:   for (const fileName of rootFilesToRefresh) {
  291:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
true);
  292:   }
  293:   for (const fileName of rootFilesToSeed) {
  294:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(workspaceRoot, fileName),
false);
  295:   }
  296:   for (const directoryName of directoryRefreshes) {
  297:     await copyDirectoryRefresh(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
  298:   }
  299:   for (const directoryName of directorySeeds) {
  300:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(workspaceRoot,
directoryName));
  301:   }
  302:
  303:   for (const fileName of portfolioFilesToSeed) {
  304:     await copyFileIfAvailable(path.join(bundledSiteRoot, fileName), path.join(portfolioRoot, fileName),
```

```
false);
  305:   }
  306:   await copyFileIfAvailable(path.join(bundledSiteRoot, "portfolio-README.md"), path.join(portfolioRoot,
"README.md"), false);
  307:   for (const directoryName of portfolioDirectoriesToSeed) {
  308:     await copyDirectoryMissingFiles(path.join(bundledSiteRoot, directoryName), path.join(portfolioRoot,
directoryName));
  309:   }
  310:
  311:   return { workspaceRoot, portfolioRoot };
  312: }
```

resolveWorkspaceRoots (main.cjs, lines 314-337)

This function decides where the builder workspace and portfolio workspace live for the current run. It combines packaged-app defaults, development-mode paths, and user-owned writable locations so the rest of the app can use stable roots instead of guessing paths repeatedly.

The builder has to work when launched from source, from an installed executable, and from an updated install. Centralizing workspace resolution prevents different parts of the app from accidentally reading or writing different copies of the portfolio.

It calls or references workspaceLooksUsable, join, dirname, resolve, findRepositoryNearExecutable, and preparePackagedWorkspace. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 317: return {; line 324: return {; line 331: return {; line 336: return preparePackagedWorkspace();.

Important local values include envWorkspace at line 315; workspaceRoot at line 323; nearbyRepository at line 329. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  314: async function resolveWorkspaceRoots() {
  315:   const envWorkspace = process.env.OMB_BUILDER_WORKSPACE;
  316:   if (envWorkspace && workspaceLooksUsable(envWorkspace)) {
  317:     return {
  318:       workspaceRoot: envWorkspace,
  319:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || path.join(path.dirname(envWorkspace),
"portfolio")
  320:     };
  321:   }
  322:   if (!app.isPackaged) {
  323:     const workspaceRoot = path.resolve(__dirname);
  324:     return {
  325:       workspaceRoot,
  326:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || workspaceRoot
  327:     };
  328:   }
  329:   const nearbyRepository = findRepositoryNearExecutable();
  330:   if (nearbyRepository) {
  331:     return {
  332:       workspaceRoot: nearbyRepository,
  333:       portfolioRoot: process.env.OMB_PORTFOLIO_WORKSPACE || nearbyRepository
  334:     };
  335:   }
  336:   return preparePackagedWorkspace();
  337: }
```

findFreePort (main.cjs, lines 339-349)

findFreePort part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 339-349 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references createServer, once, listen, address, close, and resolve. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 340: return new Promise((resolve, reject) => {.

Important local values include tester at line 341; address at line 344; port at line 345. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
  339: function findFreePort() {
  340:   return new Promise((resolve, reject) => {
  341:     const tester = net.createServer();
```

```
342:     tester.once("error", reject);
343:     tester.listen(0, "127.0.0.1", () => {
344:       const address = tester.address();
345:       const port = typeof address === "object" && address ? address.port : 0;
346:       tester.close(() => resolve(port));
347:     });
348:   });
349: }
```

startBuilderServer (main.cjs, lines 351-364)

This function starts the local backend process that serves the builder UI and handles local APIs. It chooses a port, launches server.mjs with the resolved workspace roots, and gives Electron an origin URL it can load.

The Electron window is only the desktop shell. The heavy work still belongs to the backend: saving files, running Git, compiling code, and publishing. This function is the bridge between the desktop shell and that backend.

It calls or references findFreePort, getVersion, join, pathToFileURL, and now. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 363: return `http://127.0.0.1:${port}`;.

Important local values include port at line 352; serverPath at line 361. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
351: async function startBuilderServer(workspaceRoot, portfolioRoot) {
352:   const port = await findFreePort();
353:   process.env.PORT = String(port);
354:   process.env.HOST = "127.0.0.1";
355:   process.env.OMB_DESKTOP_APP = "1";
356:   process.env.OMB_BUILDER_WORKSPACE = workspaceRoot;
357:   process.env.OMB_PORTFOLIO_WORKSPACE = portfolioRoot;
358:   process.env.OMB_BUILDER_REPOSITORY = defaultRepositoryUrl;
359:   process.env.OMB_APP_VERSION = app.getVersion();
360:
361:   const serverPath = path.join(workspaceRoot, "server.mjs");
362:   await import(`${pathToFileURL(serverPath).href}?desktop=${Date.now()}`);
363:   return `http://127.0.0.1:${port}`;
364: }
```

createWindow (main.cjs, lines 366-423)

This function creates the Electron BrowserWindow and loads the local builder interface. It configures the window behavior, menu integration, navigation safety, fullscreen behavior, and the local URL where the builder runs.

This is the point where the desktop application becomes visible. A mistake here can make the app look like a browser tab, block menus, break fullscreen, or load the wrong workspace.

It calls or references join, BrowserWindow, fileExists, setMenu, createAppMenu, setMenuBarVisibility, setAutoHideMenuBar, isDestroyed, on, preventDefault, and 6 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 402: return;; line 411: if (url.startsWith(origin)) return { action: "allow" };; line 413: return { action: "deny" };; line 417: if (url.startsWith(origin)) return;.

Important local values include iconPath at line 367; refreshFullscreenMenu at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
366: function createWindow(workspaceRoot, origin) {
367:   const iconPath = path.join(workspaceRoot, "assets", "omb-app-icon-256.png");
368:   nativeTheme.themeSource = "light";
369:   mainWindow = new BrowserWindow({
370:     width: 1440,
371:     height: 960,
372:     minWidth: 980,
373:     minHeight: 680,
374:     title: "OMB Portfolio Builder",
375:     autoHideMenuBar: false,
376:     icon: fileExists(iconPath) ? iconPath : undefined,
377:     backgroundColor: "#eef8fd",
378:     webPreferences: {
379:       contextIsolation: true,
380:       nodeIntegration: false,
381:       sandbox: true
382:     }
383:   });
384:   mainWindow.setMenu(createAppMenu(origin));
385:   mainWindow.setMenuBarVisibility(true);
```

```
386:   mainWindow.setAutoHideMenuBar(false);
387:
388:   const refreshFullscreenMenu = () => {
389:     if (mainWindow && !mainWindow.isDestroyed()) {
390:       mainWindow.setMenuBarVisibility(true);
391:       mainWindow.setAutoHideMenuBar(false);
392:       mainWindow.setMenu(createAppMenu(origin));
393:     }
394:   };
395:   mainWindow.on("enter-full-screen", refreshFullscreenMenu);
396:   mainWindow.on("leave-full-screen", refreshFullscreenMenu);
397:
398:   mainWindow.webContents.on("before-input-event", (event, input) => {
399:     if (input.key === "F11") {
400:       event.preventDefault();
401:       setBuilderFullScreen();
402:       return;
403:     }
404:     if (input.key === "Escape" && mainWindow.isFullScreen()) {
405:       event.preventDefault();
406:       setBuilderFullScreen(false);
407:     }
408:   });
409:
410:   mainWindow.webContents.setWindowOpenHandler(({ url }) => {
411:     if (url.startsWith(origin)) return { action: "allow" };
412:     shell.openExternal(url);
413:     return { action: "deny" };
414:   });
415:
416:   mainWindow.webContents.on("will-navigate", (event, url) => {
417:     if (url.startsWith(origin)) return;
418:     event.preventDefault();
419:     shell.openExternal(url);
420:   });
421:
422:   mainWindow.loadURL(`${origin}/template-preview.html`);
423: }
```

refreshFullscreenMenu (main.cjs, lines 388-394)

refreshFullscreenMenu part of electron desktop startup, window control, workspace setup, menu behavior, or update handoff. In this file, it appears around lines 388-394 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so main.cjs can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references isDestroyed, setMenuBarVisibility, setAutoHideMenuBar, setMenu, and createAppMenu. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include refreshFullscreenMenu at line 388. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
388:   const refreshFullscreenMenu = () => {
389:     if (mainWindow && !mainWindow.isDestroyed()) {
390:       mainWindow.setMenuBarVisibility(true);
391:       mainWindow.setAutoHideMenuBar(false);
392:       mainWindow.setMenu(createAppMenu(origin));
393:     }
394:   };
```

boot (main.cjs, lines 425-447)

This is the main startup routine. It prepares the workspace, starts the backend server, creates the desktop window, and connects the pieces in the order required for the builder to become usable.

A desktop app needs a controlled startup sequence. The server must exist before the window loads, and the workspace must exist before the server can serve or save data. boot keeps that sequence explicit.

It calls or references requestSingleInstanceLock, quit, on, isMinimized, restore, focus, whenReady, resolveWorkspaceRoots, startBuilderServer, createWindow, and 1 more. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 429: return;; line 433: if (!mainWindow) return;.

Important local values include gotLock at line 426. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
async function boot() {
  const gotLock = app.requestSingleInstanceLock();
  if (!gotLock) {
    app.quit();
    return;
  }

  app.on("second-instance", () => {
    if (!mainWindow) return;
    if (mainWindow.isMinimized()) mainWindow.restore();
    mainWindow.focus();
  });

  await app.whenReady();
  try {
    const { workspaceRoot, portfolioRoot } = await resolveWorkspaceRoots();
    builderOrigin = await startBuilderServer(workspaceRoot, portfolioRoot);
    createWindow(workspaceRoot, builderOrigin);
  } catch (error) {
    dialog.showErrorBox("OMB Portfolio Builder could not start", error?.message || String(error));
    app.quit();
  }
}
```

Representative opening snippet

```
const { app, BrowserWindow, Menu, dialog, nativeTheme, shell } = require("electron");
const { execFile } = require("node:child_process");
const fs = require("node:fs");
const fsp = require("node:fs/promises");
const net = require("node:net");
const path = require("node:path");
const { pathToFileURL } = require("node:url");
const { promisify } = require("node:util");

let mainWindow = null;
let builderOrigin = "";
let updateShutdownStarted = false;
const execFileAsync = promisify(execFile);
const defaultRepositoryUrl = process.env.OMB_BUILDER_REPOSITORY || "";
const defaultWorkspaceHomeName = "OMB Portfolio Builder";

const rootFilesToRefresh = [
  "server.mjs",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  "builder-rich-future-sections.js",
  "template-preview.html",
  "template-preview.css",
  "template-preview.js",
  "_headers"
];

const rootFilesToSeed = [
  "projects.json",
  "project-templates.json",
  "README.md",
  ".nojekyll",
  "robots.txt"
];

const directoryRefreshes = ["cloudflare"];
const directorySeeds = ["assets", "Backgrounds", "docs", ".well-known"];
const portfolioFilesToSeed = [
  "projects.json",
  "index.html",
  "styles.css",
  "script.js",
  "electronics-search.js",
  ".nojekyll",
  "robots.txt",
  "_headers",
  "sitemap.xml"
];
const portfolioDirectoriesToSeed = ["assets", "Backgrounds", "docs", ".well-known"];
const gitCandidates = [
  process.env.GIT_EXE,
  "git",
  "C:\\Program Files\\Git\\cmd\\git.exe",
  "C:\\Program Files\\Git\\bin\\git.exe",
  "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
  "C:\\Program Files (x86)\\Git\\bin\\git.exe"
].filter(Boolean);

function dispatchBuilderMenuAction(action) {
  if (!mainWindow || mainWindow.isDestroyed()) return;
```

```
  const payload = JSON.stringify(action);
  mainWindow.webContents.executeJavaScript(
    `window.dispatchEvent(new CustomEvent("builder-menu-action", { detail: ${payload} }));`,
    true
  ).catch(() => {});
}

function quitForBuilderUpdate() {
  if (updateShutdownStarted) return;
  updateShutdownStarted = true;
```